

«Project: Log Horizon Style JRPG» (рабочее название):[Bunker Protocol]

Жанр: JRPG / Survival Action

Движок: Собственный (C++17/20, Vulkan/DX11 через Proton)

Стиль: Log Horizon + Fallout 4/76 + Tales of Arise

1. Основная концепция

Идея и сеттинг: Техногенный мир, где границы между игровой условностью и системным кодом стерты «Игра внутри системы». Интерфейс — это часть лора (терминал выжившего). Это хардкорная JRPG на C++, вдохновленная аниме «Log Horizon» и серией «Tales of». Игра строится вокруг идеи «игры внутри системы», где интерфейс и механики имитируют работу операционной системы или глубокое погружение в виртуальную реальность.

2. Основная концепция и Мир

Старт игры: Выход из изоляции. Ключевая механика перемещения — вертикальные и диагональные лифты, соединяющие уровни бункера с поверхностью.

Старт: Выход из изоляции. Перемещение между уровнями бункера и поверхностью через вертикальные и диагональные лифты.

Масштабируемость: Мир разбит на бинарные файлы локаций (.map). Быстрая зонированная подгрузка (пре-лоадинг соседних зон).

Масштабируемость: Мир разбит на бинарные файлы локаций (.map). Подгрузка реализована по принципу Fallout (NV/4/76), обеспечивая бесшовность при перемещении.

3. Ключевые особенности (Pillars)

Системная изоляция Security Layer (Изоляция): Программный «забор». Игра не лезет в BIOS и системные папки. Основная база ставится по выбору пользователя, а логи и конфиги пишутся в AppData доступа к BIOS и критическим файлам ОС.

Гибридная камера (Fallout Style): Бесшовное переключение между видом «из глаз» (для исследования и погружения) и видом «из-за плеча» (для динамичных сражений и оценки окружения).

Уникальная идентификация (#ID): Каждый объект, моб и игрок в мире имеет строгий формат #XXXXXXX. Это не просто текст, а часть системы индексации, на которой завязана вся логика взаимодействия.

Оптимизация под Proton/Linux: Проект изначально заточивается под работу на слабых машинах через слои трансляции, обеспечивая высокий FPS при минималистичной, но стильной графике.

Hardware & Time Sync:

Сбор инфы о CPU/GPU для вывода в меню.

Тройное время: Однократная синхронизация с часами ПК при запуске. Вывод: Местное время / Время хоста (друга) / Игровое время.

Оптимизация: Приоритет — стабильный FPS на слабых машинах под Linux/Proton.

4. Визуальная составляющая (Asset System)

Прототипирование: Вместо пустых кубов используются "белые модели" (Whitebox). Они имеют финальную физическую форму (например, замок или ящик), но лишены текстур (краски, выбоин, мелких деталей).

Цветовое кодирование: Для визуального разделения объектов используется разная заливка. Ящик будет одного цвета, стены крепости — другого, чтобы игрок считывал геометрию без текстур.

Ландшафт: Временная "зеленая земля" для тестов, которая позже заменяется детальным

мешем через внешний Map Editor (аналог Creation Kit).

Smart Proxy (Заглушки): Вместо пустых кубов используются цветные примитивы (куб, сфера, цилиндр), повторяющие физическую форму финального предмета. Это позволяет считывать геометрию и хитбоксы без текстур.

Ландшафт: Временная «зеленая земля» для тестов, заменяемая на детальный меш через внешний Map Editor.

5. Технологический стек и Безопасность

Security Layer (Изоляция): Программный "забор", отсекающий игру от BIOS и системных реестров.

Hardware Sync: Движок считывает данные железа (CPU/RAM) и выводит их в игровое меню (стиль Log Horizon).

Auto-Scaling: Графика автоматически подстраивается под мощность устройства, но сохраняется "Пользовательская версия" для ручного тюнинга.

Proton/Linux: Приоритетная цель — стабильный высокий FPS на слабых машинах через слои трансляции.

6. Сеттинг и Мир

Стиль: Техно-фэнтези. Мир, где магические механики переплетены с программными терминами.

Визуал: На этапе разработки используются кубы-заглушки и шейдеры, создающие «цифровой» вид (Placeholder Art), который позже сменится на лоу-поли стилистику.

Масштабируемость: Мир состоит из бинарных .map файлов, созданных в собственном редакторе, что позволяет быстро подгружать локации без лишних затрат памяти.

7. Игровой процесс (Gameplay)

Боевая система: В духе Tales of Arise — активные перемещения, комбо и использование способностей, завязанных на характеристиках персонажа.

Интерфейс (UI): Глубоко интегрирован в лор. Меню игрока выглядит как системная консоль, отображающая статус системы и персонажа одновременно.

8. Система идентификации (Registry_ID)

Единая база для всех сущностей в формате #XXXXXXX / @XXXXXXX:

User ID: #12345 (Аккаунт игрока)

Character ID: @12345 (Персонаж в мире)

Monster ID: #%mid_12345 (Обычный моб)

Boss ID: #%midB_12345 (Босс)

Единая база для всех сущностей. Формат определяет тип взаимодействия:

User ID: #12345 — Аккаунт игрока (User ID).

Character ID: @12345 — Персонаж в мире (Character ID).

#%mid_12345 — Обычный моб.

#%midB_12345 — Босс.

#%it_12345 — Предмет/Лут.

[% XXXXXX] — Строение (Статический объект).

[#trXXXXXX] — Транспорт (Динамический объект).

9. Боевая система и Геймплей

Бой: Динамика Tales of Arise (комбо, активные уклонения) переплетенная с элементами выживания и прогрессии Fallout 76{система перков и характеристик Fallout 4/76.}
Интерфейс: Смесь системных окон Log Horizon и функциональности Pip-Boy.
Skill System: Техники с откатами (Log Horizon), завязанные на параметрах S.P.E.C.I.A.L.
State Machine (Цикл игры): Запуск -> Лаунчер (стиль Ancient Tales) -> Вход/Регистрация -> Загрузка мира с точки сохранения.
Интерфейс (UI/HUD) и Ввод

Визуал: Стиль Log Horizon (плавающие окна, статус группы). ID объектов отображаются над их головами.

Чат: Функционал Crossout (разделение на вкладки: Общий, Логи системы, Группа).

Input Handler: Одновременная поддержка Клавиатуры, Геймпада и Трекбола (эмуляция через относительное смещение осей для плавности шарика).

10. Технический стек

Язык: Чистый C++ (стандарт 17/20).

Графика: Vulkan или DirectX 11 (через обертку для Proton).

Архитектура: Модульная, с четким разделением на Engine_Core, Security_Layer и Game_Logic.

Structure

1. Ядро Системы (System/Core)

Это «прослойка» между игрой и железом.

Security_Layer: Та самая изоляция. Проверка прав доступа, чтобы игра не лезла в BIOS/системные папки.

{не нужно проверять права доступа мы записываем данные в AppData(логи и т.п.) а основа будет устанавливаться по как хочет пользователь}

Hardware_Monitor: Сбор инфы о CPU/GPU для вывода в меню (как ты и хотел).{+время на ПК но один раз только для синхронизации часов в игре для удобства в самой игре будет два времени одно местное а под ним время сервера если есть или время с ПК друга к которому подключились}

Registry_ID: Глобальный менеджер объектов. Присваивает каждому предмету, мобу или игроку уникальный #XXXXXXX.

{уже обсуждали

Единая база для всех сущностей в формате #XXXXXXX / @XXXXXXX:

User ID: #12345 (Аккаунт игрока)

Character ID: @12345 (Персонаж в мире)

Monster ID: #%mid_12345 (Обычный моб)

Boss ID: #%midB_12345 (Босс)

для вещей : #%it_12345 (предмет)

а строения отдельно я думаю вообще сделать по другому

строение [% XXXXXX] (строение)

транспорт [#trXXXXXX] (транспорт)

}

2. Движок и Рендер (Engine/Graphics)

Используем Vulkan (лучше для Proton) или DX11.

Camera_Controller: Логика «Fallout-style». Плавный зум от 1-го лица до 3-го.

Scene_Manager: Загрузка твоих .map файлов из редактора.

Asset_Loader: Система загрузок. Если текстуры нет — рисует цветной куб под форму предмета который мы создаем, чтобы игра не вылетала.

3. Игровая Логика (Gameplay/RPG)

Здесь живет душа JRPG.

Entity_Component_System (ECS): Чтобы тысячи мобов не тормозили процессор. {для каждого игрока на одной локации оно одинаковое а если они в разных локациях то mobs на второй локации для первого игрока не существуют до его перехода на нее но mobs могут перейти на локацию один из второй локации за игроком, а для второго игрока все зеркально}

Skill_System: Логика способностей (как магические техники в Log Horizon){+ скилы из Fallout 4/76 я уже писал что там и почему }

State_Machine: {тут вообще не понял типо запуск игр--> главное меню с логин формой(как обычно с возможностью регистрации просто копируем лаунчер Ancient Tales — Anime MMORPG[<https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fancient-tales.ru%2F&ved=0CBkQjhqFwoTCPjJo82525MDFQAAAAAdAAAAABAI&opi=89978449>]) --> игра с места сохранения}

4. Интерфейс (UI/HUD)

Overlay_Manager: Вывод логов{в отдельную вкладку чата остальные мы берем из аниме LogHorizon и Crossout}, здоровья и тех самых ID над головами объектов{внешка как в LogHorizon аниме там есть прямо хорошо показано но можно примотать к нему Fallout 76 и некоторые механики}.

Input_Handler: Поддержка геймпада и клавиатуры одновременно.

{здесь полностью согласен но добавлю старый стиль мыши как шарик или трэкбол а есть еще мыши с трэкболом но давай а бэззовом трэкболе остановимся его я протестировать не смогу у меня его сейчас нет}

1. Файлы, которые нужно создать/наполнить (мы их еще не трогали):

Account_System.hpp: Здесь должна быть логика хранения данных аккаунта (не персонажа, а именно юзера: почта, дата регистрации, список всех персонажей @ID).

3. Файлы, которые мы обсудили, но их нужно «собрать» воедино:

main.cpp (в папке src): Он у тебя есть на скрине, но сейчас он, скорее всего, пустой. Это главный файл, который должен подключить все .hpp, создать объект EngineCore и запустить игру.